Big Data Management — P1 Report

Cymatics: Sound-Driven Pattern Generation & Analysis Pipeline

Arman Bazarchi
Brisa Fernanda Cisneros Cervantes

Contents

1 Instructions **1**
1.1 Docker Compose (Kafka + Zookeeper + MinIO) 1
1.2 Environment Variables 1
1.3 Run Guide 2

2 Contextualization **2**
2.1 Domain and Problem Statement 2
2.2 Business Value 3
2.3 Data Sources and Categories 3
2.4 Cymatics Generation Engine 4
2.4.1 Peak Detection (Harmonic-Aware) 5

3 Implementation **5**
3.1 Landing-Zone 5
3.1.1 Ingestion paths 6

4 Architecture Design **9**

1 Instructions

Repository access: All source code and files are available on GitHub:
`https://github.com/armanbzi/BDM-Cymatics.git`

1.1 Docker Compose (Kafka + Zookeeper + MinIO)

We provide a `docker-compose.yml` file to run the local infrastructure required for our data ingestion paths.

What it runs

- **Zookeeper** (`confluentinc/cp-zookeeper:7.5.0`) on port `2181`
- **Kafka** (`confluentinc/cp-kafka:7.5.0`) on port `9092`
  - `KAFKA_ADVERTISED_LISTENERS=PLAINTEXT://localhost:9092`
  - `KAFKA_AUTO_CREATE_TOPICS_ENABLE=true`
- **MinIO** (`minio/minio:latest`) on ports `9000` (API) and `9001` (console)
  - Default credentials are defined via environment variables in the compose file

Volumes (persistence)

- `./kafka` → Kafka data directory, we store kafka files in the project's working directory in the folder"kafka"
- `./minio` → MinIO object storage data, we store our files in the project's working directory in the folder"minio"

When to run it

Run the infrastructure before executing any scripts or notebooks(unless services are already running):

```
docker compose up -d
```

1.2 Environment Variables

See `env.example` for default values. The most relevant variables are:

MinIO

- `MINIO_ENDPOINT` (default: `localhost:9000`)
- `MINIO_ACCESS_KEY` (default: `admin`)
- `MINIO_SECRET_KEY` (default: `password`)
- `MINIO_SECURE` (set to `false` for local Docker setup)
- `MINIO_BUCKET` (default: `landing-zone`)

Kafka

- `KAFKA_BOOTSTRAP_SERVERS` (default: `localhost:9092`)
- `KAFKA_TOPIC` (topic for storing event messages(warm), default: `landing-zone-events-warm-path`
- `KAFKA_TOPIC_HOT` (hot-path topic, default: `landing-zone-events-hot-path`)

1.3 Run Guide

Warm Path

```
docker compose up -d
python landing_zone_warm.py
```

Hot Path

Run in two separate terminals:

```
docker compose up -d
python landing_zone_hot_consumer.py

python landing_zone_hot_producer.py
```

2 Contextualization

2.1 Domain and Problem Statement

This project addresses the **Acoustics & Cymatics** domain, specifically focusing on transforming audio signals into visual cymatics patterns — the geometric shapes that emerge when sound vibrates a physical medium such as water or a metal plate (via sand).

> *"Given an audio signal (recorded live or sourced from datasets/APIs), generate a visual representation of its cymatics pattern (image and video showing its evolution), extract acoustic features, and organize all artifacts into a structured, queryable dataset for analysis."*

Cymatics bridges physics, music, and visual art. While the phenomenon has been studied since Ernst Chladni's 18th-century experiments, there is no large-scale, structured dataset that pairs audio recordings with their corresponding cymatics visualizations and acoustic metadata. This project aims to fill that gap by building an automated pipeline that ingests audio, generates cymatics artifacts, and stores everything in a well-organized data lakehouse.

2.2 Business Value

The pipeline delivers value across multiple domains:

- **Scientific Research:** Researchers in acoustics and physics can study frequency–pattern relationships at scale, comparing how different frequencies, amplitudes, and waveforms produce distinct geometric structures.
- **Music & Sound Design:** Audio engineers and musicians can visualize the "shape" of their sounds, enabling a new modality for understanding and comparing audio content.
- **Art & Creative Applications:** Cymatics patterns are visually striking. The pipeline can generate high-resolution artwork from any audio source, with potential applications in generative art, NFTs, and visual installations.
- **Education:** Interactive cymatics visualizations make abstract concepts like wave interference, resonance, and standing waves tangible for students.
- **Pattern Classification:** The generated dataset enables training ML models to classify sounds or visual patterns — essentially "seeing" sound.

2.3 Data Sources and Categories

The pipeline processes structured, semi-structured, and unstructured data.

Table 1: Data Sources and Categories

Category	Data	Format	Source
Structured	Audio metadata columns (frequency, amplitude, RMS, duration, file paths, . . .)	CSV / Parquet	Generated by the pipeline/Expanding source dataset (if any)
Semi-structured	Per-observation metadata (peak frequencies, spectral peaks, ...)	JSON	Generated by Kafka
Semi-structured	Streaming Event Messages when using Kafka in warm/hot paths	JSON	Generated by Kafka
Unstructured	Raw audio recordings	.wav	Live mic recording, FreeSound API, audio datasets, . . .
Unstructured	Cymatics images (peak frequency pattern)	.png	Generated by the pipeline
Unstructured	Cymatics videos (patterns appearing)	.mp4	Generated by the pipeline

Data Sources

1. **Live Microphone Recording** — the user records audio directly, and images and videos are generated and stored along with their metadata. This represents a warm/near-real-time ingestion path.

2. **Live Visualization (Hot Path)** — user is also able to visualize and see the patterns appearing and changing in real time via live microphone recording and store batches of audio observations and corresponding metadata as the microphone records (Hot Path).

3. **FreeSound / other APIs** — programmatic access to a large repository of Creative Commons audio files, enabling batch ingestion of diverse sound samples (musical instruments, environmental sounds, speech, etc.).

4. **Local Datasets** — pre-existing audio collections (e.g., UrbanSound8K, ESC-50, or custom recordings) loaded via batch processing.

One observation in the final dataset consists of:

- Raw audio file (.wav)
- Generated cymatics image (.png)
- Generated cymatics video (.mp4)
- A JSON metadata file containing: dominant frequency (Hz), all detected peak frequencies, amplitude, RMS, sample rate, duration, ...
- Streaming event messages that appeared if Kafka was used (usually JSON).
- Textual columns providing audio/visualization features, file paths (CSV, Parquet).

2.4 Cymatics Generation Engine

We implement a wave-source interference method with a **two-zone plate** and **harmonic-aware peak selection**:

1. **Two concentric zones**: an **inner disk** and an **outer ring**, each with its own wave sources and spatial scaling, to represent the pattern difference that appears via same audio vibrations and on same surface but with different water depth levels.

2. **Wave sources** are placed around zone boundaries, each source emits waves based on detected spectral frequencies. Alternating sine and cosine functions are applied between sources.

3. **Interference rendering**: the vector sum of displacements is computed on a grid and converted into a "water surface" visualization.

4. **Nodal Lines Detection:** points where displacement ≈ 0, these are where sand/particles would accumulate in a real Chladni plate, forming geometric patterns.

5. **Peak frequency**(per observation): rather than taking the single strongest FFT bin (which can be a harmonic), we pick the frequency that **appears most over time** across overlapping windows and fold harmonics down to the fundamental (energy-weighted).

6. **Rendering:** Two-color visualization with zero-crossing lines for geometric detail.

2.4.1 Peak Detection (Harmonic-Aware)

For each 5-second observation, we select the "dominant" frequency as the one that **appears most over time**, while being robust to harmonics.

High-level approach

- Split the audio into overlapping windows (0.25s, 50% hop).
- For each window, compute the FFT and extract the dominant peak (`freq_bin`).
- Build a harmonic-aware histogram by boosting **divisors** of each detected peak (e.g., 120 contributes to 60).
- Select the fundamental frequency with the strongest energy-weighted support.
- Choose the "best" window among those where the detected peak corresponds to the fundamental or its harmonics.
- Generate the cymatics image and video labels based on this selected peak.

3 Implementation

3.1 Landing-Zone

After ingesting/generating data, all data files are stored in **MinIO** (default bucket: 'landing-zone'). Kafka carries **event messages only**(references/paths), not audio files.

Paths (prefixes) used in MinIO:

- Warm path files:

- `audio/warm-path/<peak_freq>/<uuid>-<peak_freq>.wav`
- `images/warm-path/<peak_freq>/<uuid>-<peak_freq>.png`
- `videos/warm-path/<peak_freq>/<uuid>-<peak_freq>.mp4`

- Hot path files:

- **Raw capture (producer uploads here first)**: `audio/hot-path/raw/<capture_uuid>.wav`
- **consumer moves raw audio to**: `audio/hot-path/<peak_freq>/<uuid>-<peak_freq>.wav`
- `landing-zone/images/hot-path/<peak_freq>/<uuid>-<peak_freq>.png`
- `landing-zone/videos/hot-path/<peak_freq>/<uuid>-<peak_freq>.mp4`

- Metadata table:

- `landing-zone/metadata/observations.csv`

Notes

- The consumer in hot path copies raw audio into the correct folder and removes the raw object, if server-side copy fails, it re-uploads and then deletes raw.

3.1.1 Ingestion paths

Cold Path (batch, external sources) This path focuses on **batch ingestion** from external audio sources (APIs or datasets like FreeSound), using authentication tokens/credentials instead of live microphone input.

- Connect to an **external sources** via APIs (e.g., FreeSound or other datasets), authenticated using API keys or tokens.
- Fetch **batches of audio files and metadata**.
- For each audio file:
  - download the raw audio,
  - apply the **harmonic-aware peak detection**,
  - run the **two-zone cymatics generation**,
  - generate the corresponding cymatics image and video.
- Store all generated and ingested files in the landing zone under a dedicated prefix:
  - `audio/cold-path/<source>/<peak_freq>/<uuid>-<peak_freq>.wav`
  - `images/cold-path/<source>/<peak_freq>/<uuid>-<peak_freq>.png`
  - `videos/cold-path/<source>/<peak_freq>/<uuid>-<peak_freq>.mp4`
- Append records to the shared `metadata/observations.csv`, including:
  - `source` (e.g., `cold-path:<api_name>`),
  - `peak_frequency_hz`, `all_peak_frequencies_hz`,
  - `amplitude`, `RMS`, `duration`,
  - file paths for audio, image, and video,
  - external identifiers or tags from the source API.

Warm Path (near-real-time)

- Records **5 seconds** of microphone audio.
- Performs harmonic-aware dominant frequency detection.
- Generates:
  - Cymatics **video** (5s evolution)
  - Cymatics **image** (peak frequency)
- Uploads audio, image, and video to MinIO.
- Appends a row to `metadata/observations.csv`.
- Sends a Kafka metadata message after upload.

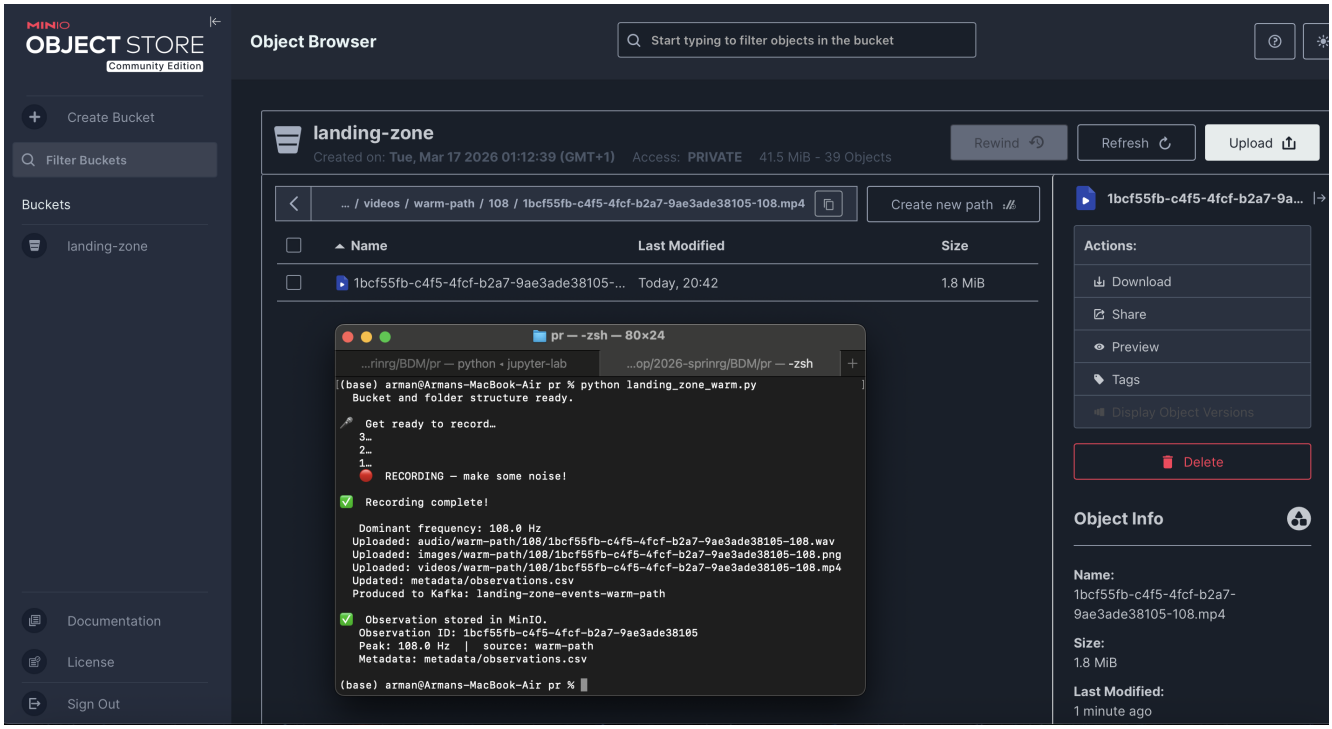

Figure 1: Warm Path Execution Example

Hot Path (producer / consumer)

Producer (live view + raw upload)

- Displays a live cymatics visualization.
- Every **5 seconds**:
  - Uploads raw audio to `landing-zone/audio/hot-path/raw/`
  - Sends a metadata-only Kafka message(including the raw audio path)
  - Flushes Kafka producer to ensure all messages recorded

Kafka message example

```
{
  "audio_path": "audio/hot-path/raw/abc123.wav",
  "bucket": "landing-zone",
  "timestamp": "2026-03-17T10:00:00Z",
  "sample_rate": 44100,
  "duration": 5
}
```

Consumer (processing + canonical move)

- Consumes Kafka messages with audio path.
- Skips already processed data using CSV + memory tracking.
- Uses manual Kafka offset commits.
- Downloads raw audio from MinIO.
- Performs peak detection and generates cymatics image/video.
- Moves audio from raw to canonical path.
- Uploads image/video and updates metadata CSV.

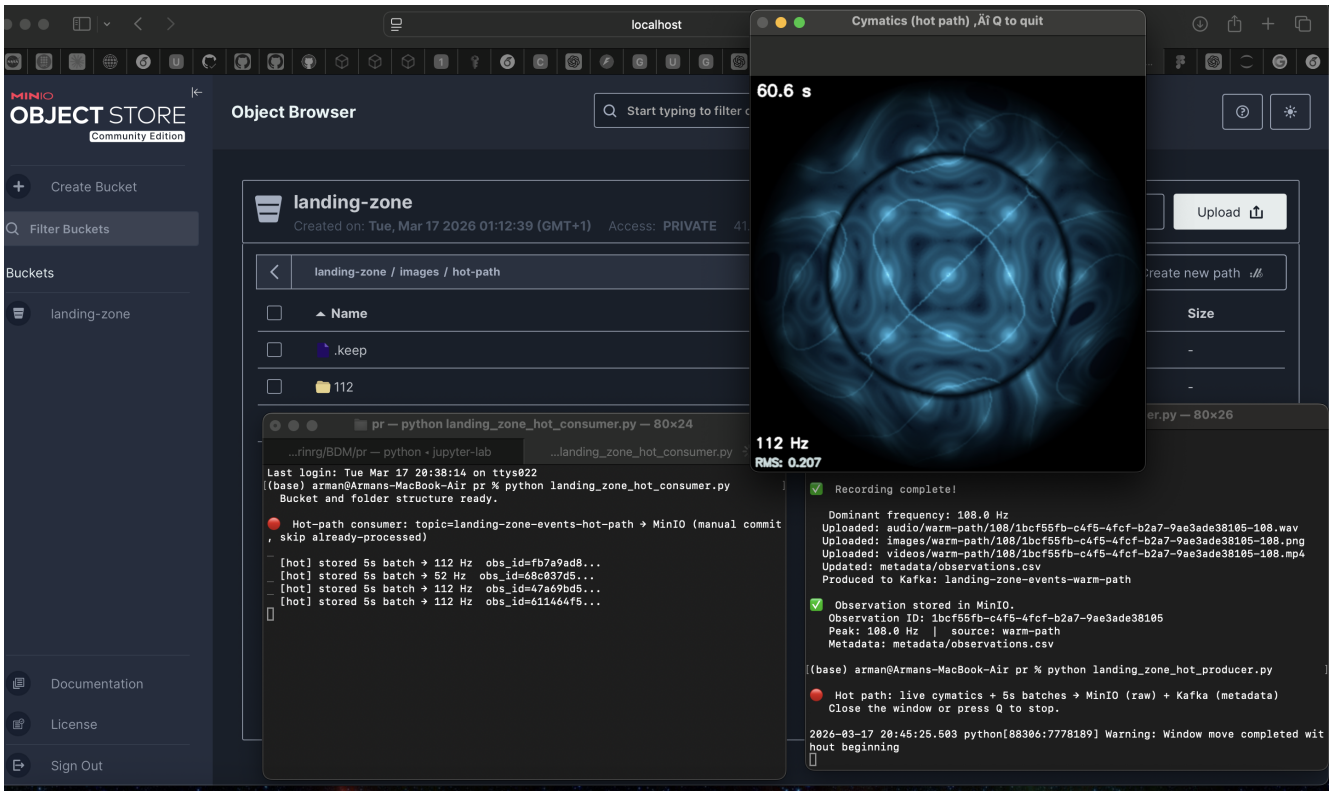

Figure 2: Hot Path Execution Example

4 Architecture Design

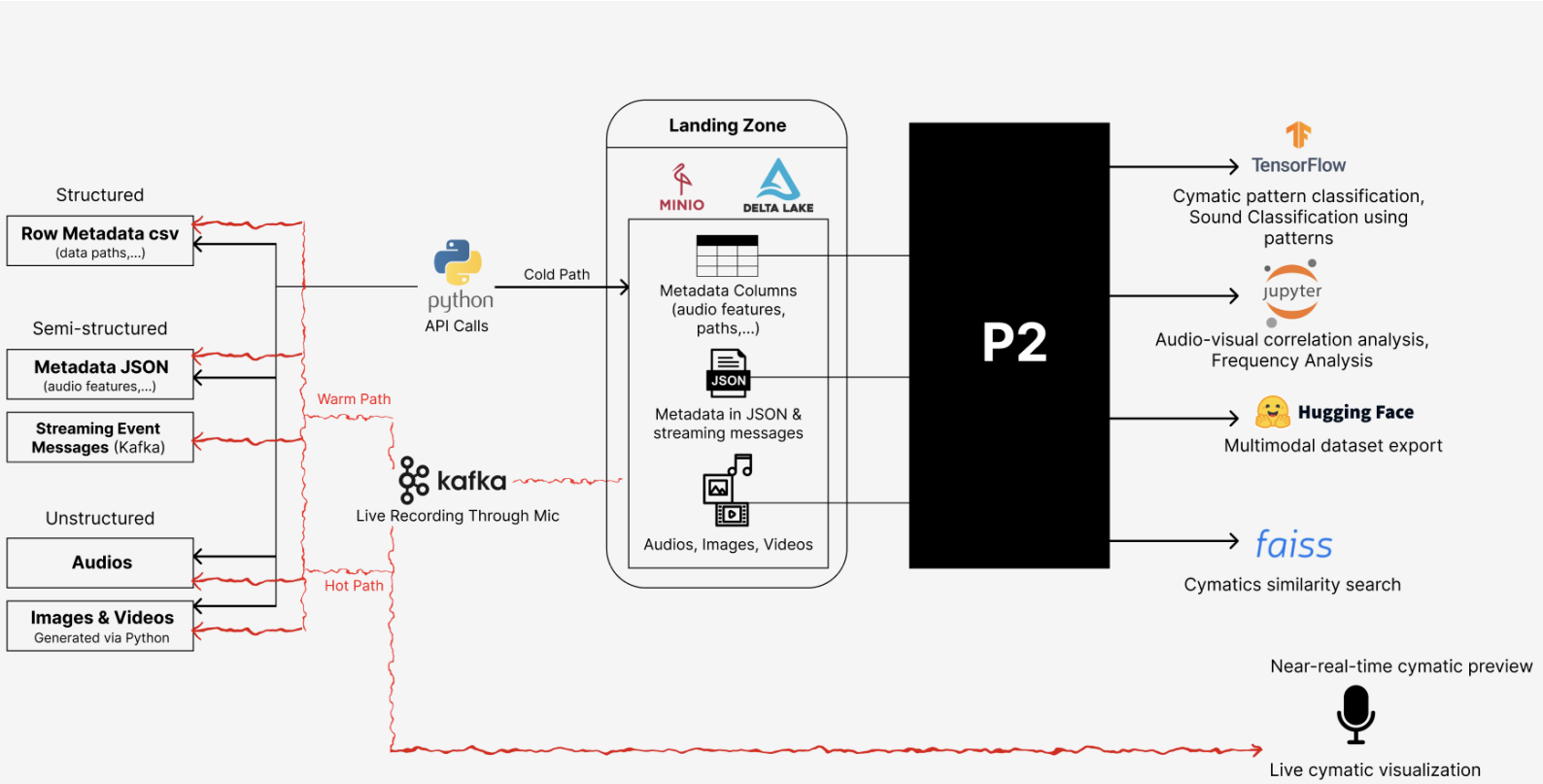

Figure 3: Architecture Design